1 · Architecture and trust boundaries



Raw note → local redaction → cleaned-text API boundary.

One note, one evidence record — The browser runs int8 ONNX OpenMed models via transformers.js. **useSynthesis.start** loads models → deidentify(raw) → extractEntities(clean) → POST clean text, spans and redaction metadata. Model-load or inference failure stops before POST. The server constructs an ExtractionResult, then writers consume it; four roles are prompts over shared facts, not four separately trained models.

Data contracts — Entity = text, type, start/end offsets, confidence, source. Offsets refer to the cleaned note. Server checks note.slice(start,end) equals entity.text; that proves occurrence, not truth, negation or timing. "Surgery considered" must remain proposed. The API accepts client text, so the local workflow alone does not enforce de-identification for every caller.

Why local extraction — Redaction before transmission limits raw-note exposure and moves inference off the server. Costs: initial download, browser RAM/CPU, device variability. A missed identifier can remain in the "clean" string. Seeded recall is not proof that every real identifier is removed.

2 · Extraction: tokens → spans → assertions

BIO aggregation — openmed.ts joins B-/I- word pieces, strips ##, finds text with a forward cursor and assigns the **minimum token score** to the span. De-ID threshold defaults to 0.5; disease/pharma to 0.6. Replace spans right-to-left to preserve earlier offsets. Disease and pharma passes run in parallel; duplicate (start,end,type) spans retain the higher confidence.

```
for span in sorted(spans, start descending):
clean = clean[:start] + tag + clean[end:]
```

Where it breaks — Reconstructed offsets can fail on punctuation, repeated mentions or tokenizer normalization. The route drops nearby negated diagnosis/symptom spans, expands abbreviations and maps chemicals through RxNorm. Nearby regex negation is not full assertion analysis: uncertainty, historical findings and family history need separate status fields.

Better test — Use tokenizer offsets where possible; fixtures for repeated terms, "no X but Y," historical vs current conditions, and proposed vs performed services. Score exact/overlap spans separately. Measure extraction and assertion errors before blaming the writer.

3 · Small-model inference and feature parity

$$\text{TF-IDF: } x_{(j)} = (1 + \ln \text{count}_{(j)}) \cdot \text{IDF}_{(j)}; \text{ normalize } x/\text{L2norm}(x)$$
$$\text{logit}_{(c)} = b_{(c)} + w_{(c)}^{T}x; \; P(c|x) = \text{softmax}(\text{logits})_{(c)}$$

classifyNoteType — Unigrams+bigrams → exported vocabulary/IDF → sparse normalized vector → class logits → stable softmax (subtract max logit). Seven note types. JSON inference avoids Python/model-server overhead, but word/feature/class order must match training exactly.

detectMissing — One logistic classifier per field uses counts, lexical flags, normalized length and one-hot note type. **present=true means the missing-field flag is active** at p≥0.5, not that documentation is present. Lexical "left" or "mg" anywhere in a note is weak contextual evidence. Compare Python/TS logits on fixed fixtures; use per-field recall and thresholds.

4 · Retrieval, reranking and the neural option



Optional neural retrieval adds validated candidates; evidence remains necessary.

Lexical path — knowledge.server.ts lazily loads compressed official artifacts. icdCandidates expands abbreviations; inverted token postings require all query tokens, then back off to fewer/strong tokens. Rank by overlap, head match and fewer extras; validate and deduplicate codes. linkDiagnoses reranks **8 candidates** using [overlap, termLen, billable, rank], keeps one, abstains below **0.15**. Literal codes overwrite lexical links.

$$\text{Rerank score} = \sigma(b + w^{T}x), \; \sigma(z)=1/(1+e\text{^}(-z))$$
Candidate recall@k bounds what any reranker can recover.

Design tradeoff — A constrained catalog makes out-of-index output impossible; a valid code can still be the wrong code. Backoff improves recall but raises ambiguity. The lexical scorer does not read the full note. Measure retrieval recall@8, reranking accuracy and coverage after abstention separately.

Neural path — BiEncoder mean-pools **masked** token states and L2-normalizes B×D vectors. Code descriptions are precomputable; query→catalog dot products retrieve candidates. CrossEncoder jointly attends to query+description for a richer score, but costs one pass per pair. Runtime /code is conditional on SYNTHURE_MODEL_API, requests top 5, validates returned codes and falls back to no neural additions on failure.

$$\text{InfoNCE logits } S = QD^{T} / \tau; \; \text{loss} = \tfrac{1}{2}[\text{CE}(S,I)+\text{CE}(S^{T},I)]$$
Mask off-diagonal same-code pairs: synonyms are not negatives.

RAG distinction — Retrieval supplies external evidence to a writer; it does not train the writer or prove entailment. Here the lexical catalog route and optional neural coder are different retrieval mechanisms. Provenance must identify actual candidates/model/version, not just say "RAG."

5 · Readiness, calibration and review

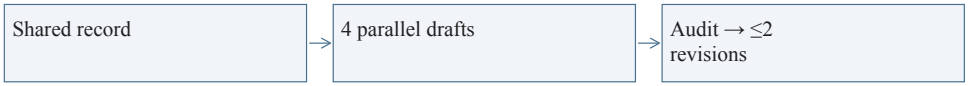
predictReadiness — Exported gradient-boosted trees traverse feature thresholds; leaf values add to initial log-odds, scaled by learning rate, then sigmoid. Isotonic knots map raw p to calibrated p. Bands: **<0.40 not ready; 0.40–<0.66 needs work; ≥0.66 ready**. These concern readiness labels, not a denial or payment probability.

$$s = \text{init} + \eta \, \Sigma_{(\text{trees})} \, \text{leaf}(x); \; p = \sigma(s)$$
Calibrated p = interpolation between stored isotonic knots.

Calibration is conditional — Fit mapping on separate data; evaluate reliability/ECE on held-out distribution. A good AUROC measures ordering, not probability accuracy. Missing evidence can block acceptance at any score. Below the first knot, this implementation can extrapolate; test boundaries and consider explicit clamping.

Knowledge and cost — ICD tabular validates codes; CCSR groups conditions; RxNorm checks drug vocabulary; CMS schedules support price illustrations; MedlinePlus adds consumer text with a 4-second timeout and per-instance cache. A fee schedule is not the patient's negotiated bill. Keep assumed deductible/coinsurance inputs visible; an inferred visit is a suggestion, not an observed procedure.

6 · Writers, audit and bounded revision



Provisional drafts stream first; final deterministic gate follows revisions.

route.ts control flow — After extraction, Promise.all launches patient, physician, hospital and employer drafts; each streams as it completes. Sonnet is selected only for hospital/physician in the frontier lane, otherwise Haiku. A forced report tool constrains structure. Combined audit waits for all drafts; flagged issues feed at most **two report revisions total**, then deterministic checks rerun.

Latency vs cost — Writer-phase time ≈ max(T1…T4), not ΣT_i; extraction, audit and revisions are serial additions. Parallel calls still incur token cost. One rejection fails Promise.all; independent failure isolation requires an explicit partial-output policy. Streamed drafts must remain provisional until final checks.

7 · Guardrails: evidence, policy, consistency

Display score = 1 − Σ(flagged severity weights)/Σ(all weights)
Decision precedence: block → escalate → revise → ship

runGuardrails — Any blocking finding wins. Otherwise extraction confidence <0.6 or no ICD/procedure codes escalates; medium/high findings request revision; otherwise ship. Layers cover input, grounding, policy, consistency, style and quality. A high average score cannot cancel a blocking finding.

What is not proven — Schema-valid JSON can be false. A number appearing in the source can be assigned to the wrong person or condition. Two agreeing model calls can repeat the same error. "Ship" is a software verdict, not downstream authorization or clinical validation. Separate supported facts, audience permission and action approval.

Stronger design — Use source-linked subject–predicate–value–status claims; test proposed→completed flips, wrong-person statistics and aggregate-view leaks. Keep human review for unsupported or high-risk outputs. Enforce gates at export/action boundaries, not just a badge in the UI.

8 · Streaming, cancellation and operations

```
buf += decoder.decode(chunk, stream=True)
while complete double-newline event exists:
parse data JSON; dispatch by event.type
```

Framing — A transport chunk may contain half an event or several events. useSynthesis buffers complete data frames and routes extracted/report/verification/safety/guardrail events. runId/alive() rejects late UI updates from superseded runs; it does **not** cancel the old HTTP request or model bill. The inspected fetch has no AbortSignal.

Production alternatives — Thread cancellation through client/server; bounded timeouts and retries; request-local errors; durable run/audit IDs. Rate-limit maps and caches are process-local, not shared across instances. Audit append is best-effort in the demo. Shared atomic quotas and durable audit storage are separate upgrades.

Measure the bottleneck — Split cold model download from warm NER, retrieval, writer fan-out, audit and revision. Record first-report latency, final-gate p95, provider errors and cost per accepted run. More agents or bigger models are not inherently better; ablate one writer vs four roles on the same inputs.

9 · Results: denominator and provenance first

Measure	Reported value / dataset
Lexical coding top-1 / top-3	74.1% / 81.6%; 727 synthetic cases
Neural acc@1 / acc@5 / MRR	40.66% / 49.18% / .4443; 3,615 CodiEsp mentions
Note type	100%; 360 synthetic; leakage caveat
Sections P / R / F1	1.000 / .696 / .821; synthetic
Missing-field micro-F1	.749; synthetic
Readiness AUROC / ECE	.839 / .054; 360 synthetic
Readiness accuracy	88.3% all → 91.7% at ~80% coverage
Seeded de-ID / disease / pharma recall	1.00 (90) / .83 (60) / .944 (60)
Adversarial cases caught	15/15 handcrafted deterministic cases

Comparability — These are ledger-reported results, not reruns. Synthetic and translated CodiEsp results are different tasks/distributions: 74.1 vs 40.66 is not a direct lexical-vs-neural comparison. Synthetic labels generated with the note can reward template recognition rather than real generalization.

Precision = TP/(TP+FP); Recall = TP/(TP+FN)
F1 = 2PR/(P+R); MRR = mean(1/rank of correct answer)

Concrete denominator — If 12 facts are present, 8 are found and 2 false ones added: precision=8/10=.80; recall=8/12=.667; F1≈.727. Perfect precision can coexist with missed facts. Report entity type, span matching policy and sample counts.

Typed evidence — Out-of-index rate 0 is **by construction**, not zero coding error. Real-note type accuracy, learned faithfulness AUROC, human fabrication rate, p95 and cost remain deferred/null in the ledger. Null means unmeasured. Top-level commit is null, so preserve the inspected repository revision with any exported results.

10 · What to defend and what to change

Architecture decision — Cheap local extraction and small exported classifiers produce an inspectable record; catalog retrieval constrains codes; hosted models narrate; deterministic and model checks govern review. Explain what each stage consumes/returns and why its failure cannot be repaired merely by better prose.

Most useful changes — Offset-aware extraction and assertion status; contextual candidate validation; grouped real-note holdouts; calibration boundary tests; per-stage failure states/cancellation; durable gate-enforced exports. Compare each against the current baseline rather than claiming an unmeasured improvement.

Reproduction checklist — Pin code/data/model artifacts and feature order. Verify TS/Python parity, reload artifacts, test absent fields/negation and provider failure, freeze train/calibration/test roles, then compare accepted quality *and* coverage. Measure resource/cost effects before scaling.

Source map — OpenMed: lib/openmed.ts; orchestration: lib/useSynthesis.ts; API: app/api/synthesize/route.ts; lexical knowledge: lib/knowledge.server.ts; predictors: lib/models/synthure.ts; neural coder: ml/icd_coder/model.py; gates: lib/guardrails.ts; metrics: data/evals.json. Blog sections 1–13 cover the full derivations and pinned code excerpts.

1 · Runtime architecture and data contracts



Fact source, evidence selection and prose generation are separate stages.

Request trace — GET /api/games/{id}/recap → local/Redis cache → ESPN game+summary → normalized facts → key-play selection → writer or template → labeled response. Keep fetching, classification, ranking, writing and persistence distinct. The live path is not automatically the historical PostgreSQL ingestion path elsewhere in the repo.

espn_public.py — _event_to_game requires competition, home/away participants and event ID; outputs id, sport, status, date, teams and nullable scores. _score_value returns None for missing/invalid values. A missing score is unknown, not zero; zero is a valid observed score. _get_json uses URL+query key, timeout=12s and raises for HTTP failures.

Adapter boundary — Normalize provider-specific nesting once so downstream code consumes stable fields. Fixtures should cover missing participants, invalid IDs, postponed/live/final games and corrected scores. Preserve source timestamp and event identity; a fresh HTTP response is not evidence that upstream facts are final.

2 · Cache semantics and concurrent misses

Layer	Behavior / boundary
Source JSON	Process-local URL+query cache; lock around access
Recap memo	recap:v2:{game_id}; six-hour TTL; size clear
Redis	Shared when configured; separate from local memo
Eligibility	Both scores, not live, and LLM success or no LLM configured

Current behavior — get_recap already avoids caching a data fallback when an LLM is configured. That prevents a transient provider failure from pinning the inferior response. Memo and source locks protect get/set, but release before fetching/generation: simultaneous misses can still duplicate expensive work.

```
hit? return hit
miss -> fetch -> build -> eligibility check -> cache
# No per-key single-flight covers this entire path.
```

What to improve — Key lacks source revision, prompt and model revision. Corrected final data can reuse stale prose. Add a fact fingerprint + schema/prompt version; coalesce work per key; use explicit stale/refresh policy. Cache shared game facts separately from personalized results. Compare hit/miss latency and cost, not one blended average.

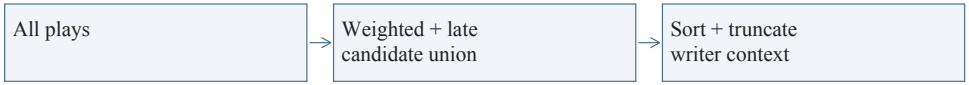
3 · Play analysis: rules behind the agent names

Stable interface — PlayContext holds sport, description, period/clock, current/previous scores and optional field position. analyze(ctx) returns AgentRead with headline/detail/metric. Scout, Stat and other modules are deterministic rules, not trained CV models. The contract allows later replacements with independently tested learned implementations.

$$\text{Kick proxy } p = 1/[1+\exp((\text{distance}-53.5)/6.5)]$$
$$\text{Win proxy} = \sigma(\text{margin} / \max(.5, 7\sqrt{\max(.02,\text{time_left/total})}))$$

Limits visible in code — Hand-set sigmoid constants are not calibrated probabilities. Missing kick distance returns 0.0. Some Scout text asserts tactical causes absent from the event record; Stat can substitute 2/3 points when score_delta is zero. Determinism does not guarantee truth. Test missed shots, overtime and missing clock/distance; return unknown instead of fabricated certainty.

4 · Evidence selection and recap generation



Truncating the union can drop a late decisive event; reserve its slot first.

_key_plays — Category weights include touchdown=10, dunk=9, three-pointer/interception=8; "miss" reduces weight. It unions highest-weight and late plays, deduplicates by Python object identity, sorts by ID and truncates. The final truncation can remove late plays it intended to preserve. ID ordering is an assumption about chronology; object identity is not a stable event ID.

Why select first — Selection compresses the prompt but can remove the decisive evidence. The writer receives _game_facts: teams/scores/date/status, period scores, lead changes, key plays and leaders. Improve with reserved late-event slots, explicit period/clock ordering and stable-ID deduplication; evaluate selection recall separately from prose quality.

Writer paths — General recap tries configured Anthropic models, then OpenAI, then _data_recap. generated_by labels llm vs data; response also exposes provider failure when available. Both scores are used to infer played status because the source status can be mislabeled. This is a heuristic, not proof a game has ended.

Grounding requirement — Every sentence needs a supported relation, not merely familiar names/numbers. If A has 30 points and B has 12, "B scored 30" is false even when all tokens occur in facts. A template reduces some freedom but can still misstate missing/tied/live data.

5 · Reels, two-host broadcasts and timing

Media architecture — Existing source video URLs supply footage; scene metadata, overlays and generated narration arrange it. Speech synthesis is text→audio; speech recognition is audio→text. Neither means footage was generated or tactical events were visually recognized. Missing clips should stay unavailable.

broadcast.py — Two hosts: play and analyst. Requested tiers: 120/300/600 seconds with 12/26/44 turns; budgets 2400/5200/8000 tokens. Turn schema: host, text, scene_type, duration_hint. Conditional trained writer → Anthropic fallback → deterministic broadcast. A turn count is a planning target, not measured spoken duration.

```
{host: "play", text: "...",
scene_type: "moment", duration_hint: 8}
```

Parsing and failure — _extract_turns parses the array, then salvages flat JSON objects if truncated. _clean_turns normalizes enums, cuts text at 400 chars, clamps hints to 4–20s; malformed int conversion can still fail. Regex salvage mishandles nested structures/braces in strings and cannot prove coherence or truth.

Better contract — Use schema-constrained output, evidence IDs, safe validation and explicit partial-result handling. The analyst prompt requests causal tactics that event descriptions may not support. Permit "not observed." Synchronize scenes to real audio completion; cancel stale speech before follow-up playback. Validate chronology, facts, media and timing independently.

6 · Personalization before writing

$$s = \sigma(b + w^T x);\ x=[\text{team, player, recency, magnitude, upcoming}]$$
$$\text{recency} = \exp(-\max(\text{age_days},0)/7);\ \text{half-life} = 7 \ln 2 \approx 4.85\ \text{d}$$

curation.py — Default bias −0.5; heuristic weights 2.2, 2.0, 1.3, 1.1, 0.6. Clip magnitude to [0,1]; team/player/upcoming are indicators. _weights loads curation_weights.json or heuristic-v1 and memoizes once per process. No exported weight file was found in the inspected tree. A version label does not establish learned provenance.

Ranking consequences — Sigmoid preserves raw-score order but does not calibrate click probability. At seven days recency≈.368, not .5. rank_items rounds to four decimals before sorting, which can create ties. Sort full precision, round for display. Newsletter then writes ranked evidence; a better writer cannot restore an omitted followed-team event.

Learned alternative — Collect exposure and interaction logs, handle cold-start fans explicitly, deduplicate game IDs, split by time/user as appropriate, compare NDCG to recency/popularity baselines. Synthetic labels from hand-chosen weights only recover those assumptions; they do not establish real-fan uplift.

7 · Distillation, LoRA and deployed evidence



Current training uses random records; grouped splitting is the improvement.

Offline pipeline — Facts → teacher drafts → grounding filter → chat-formatted pairs → adapter training → held-out evaluation → configured endpoint. Training files, saved weights, serving configuration and actual request logs establish different facts. Newsletter and broadcast have optional trained routes; do not claim every recap uses a fine-tuned model.

$$W' = W + (\alpha/r)BA;\ A{:}r\times d_{in},\ B{:}d_{out}\times r$$
$$\text{Trainable count } r(d_{in}+d_{out}),\ \text{versus } d_{in}\cdot d_{out} \text{ for full } W$$

Actual recipe — train_writer_lora.py: r=16, α=32, dropout=.05; q/k/v/o and gate/up/down projections; default lr=2e−4, 3 epochs, batch=1, accumulation=16, max sequence=4096, bf16, gradient checkpointing. For 4096² matrix: 131,072 adapter params vs 16,777,216 full (~0.78%). Checkpointing saves activations by recomputing; accumulation raises effective batch.

QLoRA and capacity — Optional NF4 4-bit frozen base, double quantization, bf16 compute; adapters still train. Saves weight/optimizer memory, not all activations or inference cost. Low rank limits update capacity; compare validation quality and throughput against LoRA/full tuning before choosing.

Leakage and filtering — Script uses random 95/5 records (seed 7), not game/time groups. grounding.check ignores numbers <13 and years, accepts either name token and allows one violation. It is a weak screen, not a guarantee. Split by game before generation; use a final time-disjoint test; check player–stat relationships and compare base LLM/template/adapter on identical facts.

Source map and recall — api/espn_public.py, play_analysis.py, recaps.py, reels.py, broadcast.py, curation.py, newsletter.py; db/store.py; training/grounding.py and train_writer_lora.py. Reconstruct the system as **facts → normalized records → selected evidence → labeled prose/media → validated state**. Blog sections 1–14 contain detailed examples and pinned excerpts.

8 · Identity, storage and the award race

Time	Request A	Request B
1 / 2	reads unresolved	reads unresolved
3 / 4	marks; awards +10	marks; awards +10
Atomic transition	check + mark + award	sees resolved; +0

Data model — Browser UUID in local storage is sent as X-Device-Id. Clearing storage or changing browser changes identity; a client-supplied ID is not authentication. Redis hashes store picks/profiles, sorted sets rank points, and saved newsletters use share tokens with expiry. Missing REDIS_URL makes storage operations no-ops, not durable success.

mark_resolved — Reads pick → checks unresolved → writes resolved → increments profile/streak and leaderboard separately. Two callers can both pass the check. A crash after marking resolved but before awarding can lose an award on retry. Individual Redis commands are atomic; the multi-command transition is not. The “exactly once” docstring overstates the implementation.

Correct alternative — Atomic Lua/transaction with conflict retry, or an idempotent award ledger plus reconciliation. Use device+game as logical operation identity and protect all counters in the same transition. Test concurrency and crashes at write boundaries. Distinguish unavailable store from already-resolved/no points.

9 · Evaluation: locate the error, then measure it

Stage	Measure / adversarial example
Adapter	Schema fixtures; missing score ≠ zero
Classification	Per-class precision/recall; missed shots
Selection	Key-event recall; decisive late play omitted
Writing	Supported claim rate; swapped player statistics
Broadcast	JSON + factuality + real playback timing
Curation	NDCG on fixed labeled candidates; cold start
State	Concurrent award, crash recovery, reload durability

$$\text{Precision} = TP/(TP+FP);\ \text{Recall} = TP/(TP+FN)$$
$$DCG = \Sigma(2^{\wedge}(\text{relevance})-1)/\log2(\text{rank}+1);\ NDCG = DCG/\text{ideal DCG}$$

Evidence boundary — No production factuality rate, trained-ranker uplift, cache hit rate or p95 is established by this review. Grounding-filter acceptance is not a quality metric: wrong relations and small fabricated numbers pass. Audit false accepts and false rejects on independently labeled claims.

10 · Serving decisions and alternatives

Bottlenecks — Source fetch, blocking model calls, cold cache and shared store can dominate. The async generation endpoint calls a synchronous recap function; blocking work can stall its event loop. _last_llm_error is module-global and can mix concurrent diagnostics. Use request-local error values and a suitable thread/job path.

Scale deliberately — Bound concurrency/retries; jitter retries; single-flight popular keys; separate live/final freshness; track source→selection→writer→store timings. GPUs are not the first fix for redundant HTTP or generation calls.

1 · Scientific question and representation contract



Same object identity is the bridge between different feature spaces.

Question — Do frozen models organize the **same galaxies** similarly across surveys, and does that geometry support the **same unchanged physical predictor**? Optical/infrared/spectral observations measure different aspects of one object. Similar neighborhoods are an observational relationship; physical transfer is a separate test.

Shapes — Images B×C×H×W → patches/tokens B×T×D → pooled B×D. Sequence adapters accept B×C×T; tabular B×F. Output: embed_for_mode→B×D; layerwise→ordered {stage:B×D}. Coordinates are learned features, not automatically mass, redshift or size.

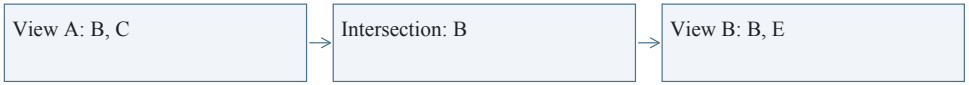
Hook code — ModelAdapter._capture_module_outputs installs selected forward hooks, accepts first tuple tensor, pools, float-casts/detaches, runs under no_grad and removes hooks in finally. Generic pooling averages spatial axes for 4D and tokens for 3D. Prevents leaked hooks; rank alone cannot identify batch axis or exclude padding/special tokens.

Modality tradeoff — fit_length right-pads with zeros+mask or keeps first T samples. Chronos takes channel 0 and mean-pools encoder tokens: no multichannel fusion is implied. Real light-curve/catalog wiring remains separate from an adapter smoke test.

2 · Cosine, neighbors and local agreement

$$\cos(x,y)=x^{\mathsf{T}}y/[\mathrm{L2norm}(x)\ \mathrm{L2norm}(y)]$$
$$Z_norm[i]=Z[i]/[\mathrm{L2norm}(Z[i])+\varepsilon];\ \mathrm{MKNN}=(1/N)\ \Sigma_i\ |N_A(i)\cap N_B(i)|/k$$

Concrete meaning — [1,2,3] and [2,4,6] have cosine 1 because proportions match. For one galaxy, neighbor lists [B,C] and [B,E] share one of two IDs: overlap=.5. Compare within-view neighbors, then identities across views; d_A may differ from d_B. "Mutual" here means cross-space overlap, not reciprocal edges.



For galaxy A: |intersection|/k = 1/2. Compare object IDs, not coordinates.

Chance and preprocessing — Uniform independent neighbor lists excluding self have expected overlap fraction k/(N−1). Optional global-percentile clipping suppresses activation outliers but changes geometry. CPU mknn signature defaults k=10, truncate_percentile=100 (disabled), despite a stale docstring mentioning 95. Paper uses k=20; record the actual setting.

Failure controls — Zero vectors have no meaningful direction; epsilon merely avoids division by zero. Reject/mark zero and nonfinite rows with a common paired mask. Explicitly exclude self, define ties and compare CPU/GPU neighbors on a hand-checkable fixture. Self-MKNN=1 and shuffled-pair controls test different failures.

3 · Depth: state versus update

Stage semantics — State h_l is accumulated representation; update Δh_l=h_l−h_(l−1) measures change. Subtract only compatible widths/coordinates/pooling. Blocks hook coarse modules; residual hooks non-leaves; leaves hooks terminal operations. Named-module traversal order is not proof of a sequential residual stream. Save stage names, execution order and shapes; more inspected stages increase selection opportunity.

4 · Pairing, splits and geometric alternatives

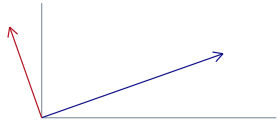
Identity before shape — Matched surveys must share unique galaxy IDs and exact row order. Extractor accumulates paired modes and drops columns whose row count differs from the majority (catches token-vs-batch errors). Equal row counts do not prove correspondence. The shown Parquet writer does not explicitly add object IDs/manifests: persist IDs, filter masks and revisions as a strengthening.

$$\text{Linear CKA} = \|X_c^{\mathsf{T}}Y_c\|^2_F\ /\ (\|X_c^{\mathsf{T}}X_c\|_F\ \|Y_c^{\mathsf{T}}Y_c\|_F)$$
$$X_c = X - \text{column means};\ Y_c \text{ likewise}$$

Metric distinctions — MKNN: local identity-neighborhood agreement. CKA: centered relational similarity, invariant to orthogonal transforms/isotropic scale. Procrustes: fits an orthogonal alignment. Exact-object retrieval: does the true paired object rank highly? RSA correlates pairwise distances; Jaccard divides neighbor intersection by union. These answer related but nonidentical questions.

Implementation cost — kernel.cka builds N×N centered Gram matrices (linear/RBF options). GPU worker uses feature-space linear CKA; denominator reused over permutations. Intermediate sizes include d_A^2, d_B^2 and d_Ad_B: favorable when d much less than N, expensive for wide output heads. Default worker excludes lm_head and d>16,384; exclusion is unavailable, not zero.

5 · Prediction, transfer and learned adaptation



x=[2,1] → R(x)=[−1,2]
Fixed predictor "read coordinate 1": 2 → −1.
Rotate **every** object: angles/neighbors survive; the old readout need not.

$$\text{Ridge: minimize } \|Xw+b-y\|^2 + \lambda\|w\|^2 \text{ (Euclidean norms)}$$
$$R^2 = 1 - \Sigma(y-\hat{y})^2\ /\ \Sigma(y-\bar{y}_{test})^2$$

Readout test — Fit preprocessing and predictor on allowed source rows; choose settings/stage on validation roles; apply unchanged to the target test. R^2=−1.5 means squared error is 2.5× the test-mean baseline, not negative accuracy. A rotation preserves angles/neighbors but changes a fixed coordinate readout.

Code vs paper — physics.linear_probe L2-normalizes, optionally fits PCA inside each fold, then uses OLS; returns mean/std/folds despite a float annotation. That is within-view decodability, not automatically the paper ridge transfer protocol. Its PCA cap uses full N rather than fold N, so requested components can exceed training-fold rank.

Adaptation controls — PCA retains high variance; CCA finds cross-view correlated directions; Procrustes learns a rotation/reflection. Fit on training pairs only. Target refitting or learned alignment is adaptation, a different claim from unchanged transfer. High-dimensional CCA can overfit; tune dimension/ridge on validation, test once, and compare shuffled-pair/random-feature/within-view baselines.

Leakage rule — Galaxy-level grouping, shared row masks, frozen preprocessing and disjoint fit/selection/test roles matter more than a seed. Never fit PCA/normalization statistics on the full test-containing matrix when the protocol requires training-only fitting. Report sample counts after filtering.

6 · Permutations and the searched maximum

$$p = (1 + \#\{\text{null} \geq \text{observed}\}) / (B + 1)$$
Worker calibrated score = clip[(observed−τ)/(1−τ), 0, 1]

Graph reuse — Build both neighbor graphs once. For permutation π, relabel graph B as π^(-1)(N_B(π(i))). Both query rows and neighbor IDs must change; shuffling rows alone leaves mismatched labels. This keeps within-view geometry while destroying cross-view correspondence and avoids repeated pairwise search.

```
perm_inv[perm] = arange(N)
B_shuffled = perm_inv[B_neighbors[perm]]
```

Selection correction — The worker calibrates each layer pair separately using pair-specific seeds. Taking its best cell is **not** a maximum-over-depth permutation test. To test a searched peak, apply the same search to each shuffled curve with a common permutation across depth, then compare maxima. Paper: k=20, B=9,999 depth-max permutations; worker defaults: k=10, B=1,000 (minimum p≈.001).

7 · Extraction and artifact engineering

extract_all_layers — Uses no_grad, collects CPU tensors per stage/mode, concatenates and writes wide Parquet. Precomputed DESI/SDSS embeddings bypass model forward. Saved embeddings let metrics/probes change without repeating inference. But this accumulates all batches before writing; it is not a bounded streaming writer.

Stored float32 activation bytes ≈ 4N Σ_(stages)d_l
Include both views, concatenation copies and metadata in RAM estimates.

Worker cache — Download one Parquet to local scratch; read selected columns. OrderedDict LRU defaults to 64 decimal GB and keeps at least two columns, so that budget can be exceeded by two large arrays. Persist model/checkpoint/data/code revisions, IDs, pooling, stage map, dtype, seeds and metric settings. Streaming row groups/shards can bound host RAM.

8 · GPU memory and exact-search complexity



Float32 scores only. One retained N×20 int64 graph: 16 MB.

_build_knn_chunked — Compute C×N cosine scores; mask local row/global self column to −∞; topk IDs into N×k int64 table; discard scores. _mknn_overlap_sparse broadcasts to N×k×k booleans, then counts matches. Storage is sparse; equality work still grows with k².

Worked budget — N=100,000, C=4096, k=20: full float32 scores 40 GB; chunk 1.6384 GB; one int64 graph 16 MB; two graphs 32 MB; N×k² Boolean intermediate ~40 MB. Embeddings/top-k workspace/CKA add more. Exact neighbor search still O(N²d); chunking saves peak storage, not quadratic arithmetic.

Alternatives — Smaller chunks reduce GPU peak but increase launches. Exact FAISS flat search is still exact; approximate indices need recall@k and MKNN-bias checks against exact subsets. TF32 can alter close-score rankings: compare strict float32 stability. For large k, sorted-set intersections can avoid k² matching.

9 · Workers, failure recovery and resource claims

Task lifecycle — Discover survey/model → claim → download → load columns → compare/validate → write temp → rename → upload done → release claim. SLURM example requests 1 GPU, 8 CPUs, 512 GB host RAM, 24h. Independent pair partitions flatten i×n_B+j; partial files need gap/duplicate checks before merge. This is task parallelism, not model sharding.

Guarantees — Local O_CREAT|O_EXCL claims atomically on a shared filesystem; stale reclaim can race with a live old owner. Remote running-marker upload is not compare-and-swap. PID-specific temp then rename prevents partial local results, but does not make remote completion exactly once. Add lease fencing/ownership, unique attempt IDs, checksums and idempotent publication.

10 · Reported findings and how to read the figures

Paper-linked result	Interpretation
36 models / 11 families; 139/139 comparisons	Significant local agreement somewhere in depth
46 stronger update peaks; 71% earlier update peak	Updates differ from accumulated states; not causal proof
8/8 geometry-selected image transfer tests worse	Geometry selection insufficient for best held-out R²
31/31 terminal redshift transfers R²<0	Unchanged readout fails test-mean baseline
Within-view mean R² .503	Useful decodable signal within a survey
Cross-view median −1.43; mean −2.98	Coordinate/readout transfer is a harder requirement

Figure reading — Fig.1: state/update curves over normalized depth; final layer need not peak. Fig.2: geometry–prediction association depends on target; best-vs-best correlation is not a selection policy. Fig.3: negative geometry-selection regret and negative cross-survey readout despite positive within-view prediction. Counts are retained from paper-linked notes, not recomputed here.

11 · Contribution, interpretability and reproduction

Documented contribution — ARAVIND.md attributes time-series/spectrum/tabular adapters and CPU-friendly device/determinism support. Real-weight CPU checks reported for Chronos, PatchTST/PatchTSMixer, Moirai and Spender. MOMENT dependency conflict; TabPFN gated weights; 39 stub tests reported. Dataset wiring remained. Stub interface test ≠ real-weight smoke test ≠ scientific benchmark.

Causality boundary — SAE learns sparse reconstruction features; crosscoder relates features across models; activation patching intervenes on internal values. Geometry correlations alone do not establish a causal mechanism. Separate personal implementation, experiments actually run, and team-level conclusions; historical speedups need their own logs/baselines.

Runbook — Pin inputs/environment → test adapter shapes → real-weight smoke check → assert paired IDs → tiny extract/reload → hand-check neighbors → self/shuffle controls → freeze selection roles → measure RAM/time → scale with restartable tasks. Seeds do not fix wrong row order, nondeterministic kernels or ties.

Source map — models/base.py, models/timeseries.py, experiments_layerwise.py; metrics/neighbors.py, kernel.py, physics.py; experiments/platonic/compute/pu_solve.py and slurm/pu_solve.sh; ARAVIND.md. Blog sections 1–19 retain derivations, figures, examples and 14 code walkthroughs.